DL Optimization and hyperparameter tuning

Michalis Vazirgiannis
Data Science & Mining group
LIX @ Ecole Polytechnique, IPP
http://www.lix.polytechnique.fr/dascim/

November 2025

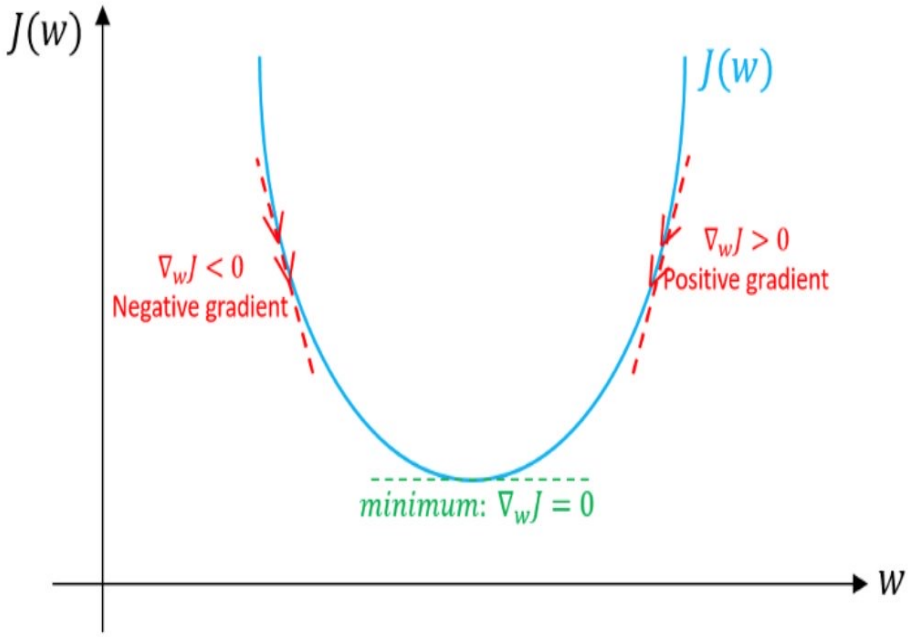
HYPERPARAMETERS IN MACHINE AND DEEP LEARNING

Error Optimization – gradient descent

Learning & Optimization: Assume $J(\theta)$ the objective/error function, θ parameters (i.e. for $y=ax+b$, $\theta=(a,b)$)

Objective: find θ that minimizes $J(\theta)$

- update the parameters in the opposite direction of the gradient of the objective function: $\nabla_\theta J(\theta)$ w.r.t. to the parameters
- Batch gradient descent
- η learning rate
- *Redundant computations:* re-computes gradients for similar examples before each parameter update.



$$\theta = \theta - \eta \bigtriangledown_\theta J(\theta)$$

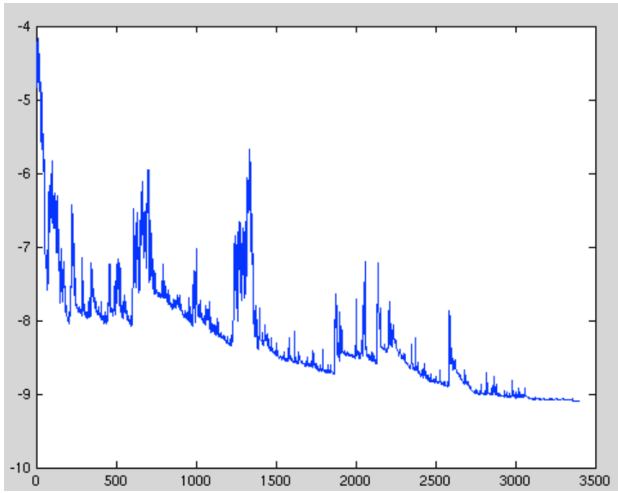
Gradient Descent Variants

How much data do we use for each update step?

- Batch Gradient Descent: $\theta = \theta - \eta \bigtriangledown_\theta J(\theta)$

  Use the entire training dataset for each update.

  Accurate gradient, but very slow on large datasets.



- Stochastic Gradient Descent (SGD):

  Use a randomly chosen example for each update.

  Noisy gradient, but much faster updates.

$$\theta = \theta - \eta \bigtriangledown_\theta J(\theta, x(i), y(i))$$

- Mini-batch Gradient Descent:

  Use a small batch of examples (e.g., 32, 64, 128).

  stable and computationally efficient. The most common approach.

Closed-Form Solutions

Sometimes, we can solve for the optimal parameters directly without iteration.

Linear Regression with MSE

optimal weights $w*$ can be found directly with the Normal

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

- When possible: Direct solution, no learning rate to tune.
- When not possible: Computing $(X^T X)^{-1}$ is very expensive for large datasets ($O(d^3)$).
- In these cases, we must use iterative methods like Gradient Descent.

Hyperparameters vs. Parameters

crucial to distinguish between these two:

Parameters

- Learned from the training data - output of the training process.
- Example: $\hat{y} = \mathbf{w}^T\mathbf{x} + b$ weights **w**, bias **b**.

Hyperparameters

- Set by the user before training begins - configure the learning process itself.
- Example: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla\mathcal{L}(\mathbf{w})$ learning rate α
- Other: regularization λ, choice of model, number of neighbors k in k-NN.

Tuning

- -use validation set to find the best hyperparameters - Grid Search or Cross-Validation.

HP importance: control speed, stability, and accuracy.

Deep Learning - Hyperparameters tuning

- consider a deep neural network on a training dataset $\{(x_i, y_i)\}_{i=1,\ldots,N}$

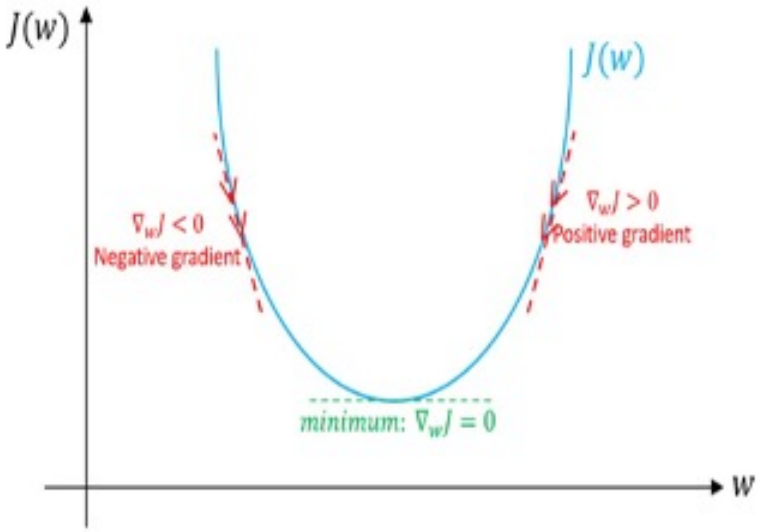
$$F(\theta, x_i) = y_i \text{ and } L(\theta) = \frac{1}{N} \sum_{i=1}^{N} l(y_i, y_i)$$

- Parameters $\theta = w$: parameters of the network, F: function representing the network, L : loss funtion of the network and l: loss for each training point.
- **Batch Stochastic Gradient Descent (SGD)** - general algorithm to train deep neural networks
- At each iteration t: $\theta_{t+1} = \theta_t - \eta \nabla_\theta \hat{L}_t$

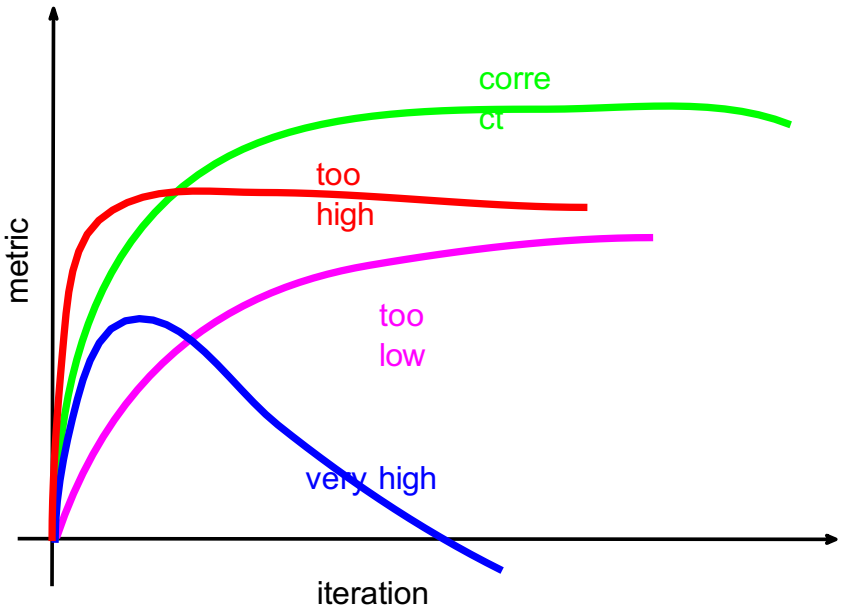
- Where η : learning rate, $\hat{L}_t = \frac{1}{B} \sum_{i=1}^{B} l(y_{b_i}, \hat{y}_{b_i})$, $\{b_1, \ldots, b_B\}$ indices of the elements in the batch.

Deep Learning - Hyperparameters tuning - Learning Rate Schedules

- In vanilla SGD learning rate is fixed, this version is still used
- too high → diverges, too low → slow convergence.
- Typical values: $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$.



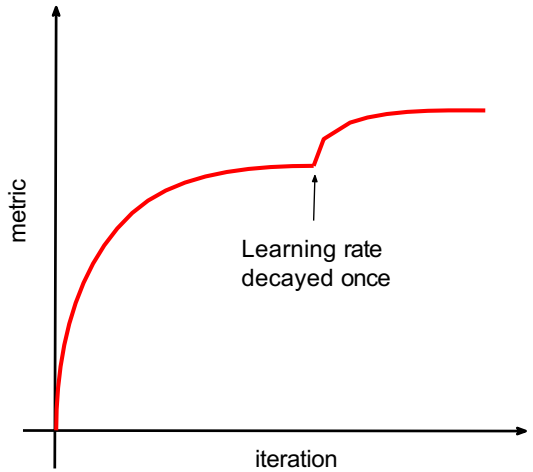
$$\theta = \theta - \eta \nabla_\theta J(\theta)$$



How to evaluate the learning rate.

Deep Learning - Hyperparameters tuning - Decaying the Learning Rate

- Covergence of training curve due to the fact that the learning rate is high to reach an optimum. i.e. θ oscilates arround an optimum.
- Decaying the learning rate once (by an order of magnitude) allows to reach closer to this optimum.



Step decay of the learning rate once

- Learning rate can be decayed on schedule (after a predefined number of iterations), or when the network stops learning

Deep Learning - Hyperparameters tuning - Learning Rate Annealing / Step-Wise Decay

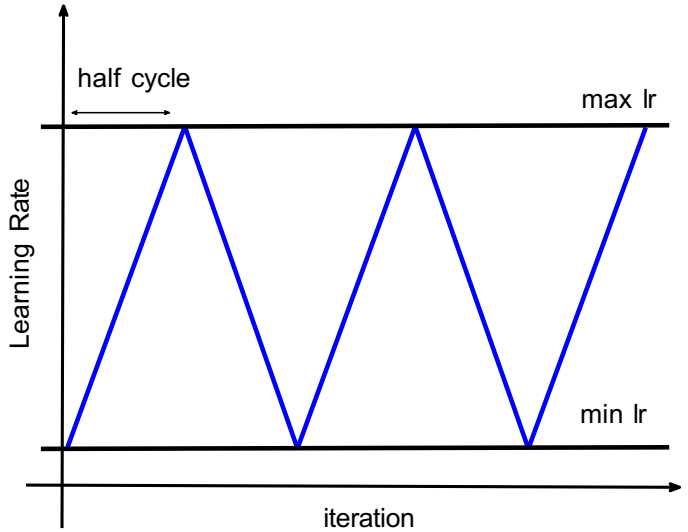
- Learning rate annealing: continuous changes of the learning rate during training.

$$\eta_t = \frac{\eta_0}{1 + decay*t}$$

- : For example $decay = 10^{-6}$. The point is to reach a minimum for the loss as fast as possible.
- Variants: exponential decay $\eta_t = \eta_0 e^{-decay*t}$.

Deep Learning - Hyperparameters tuning - Triangular Schedule

- Sequentially increase and deacrease the learning rate.
- Intuition: increasing the learning rate allows to escape local minima, deacreasing it allows to correctly train the network.

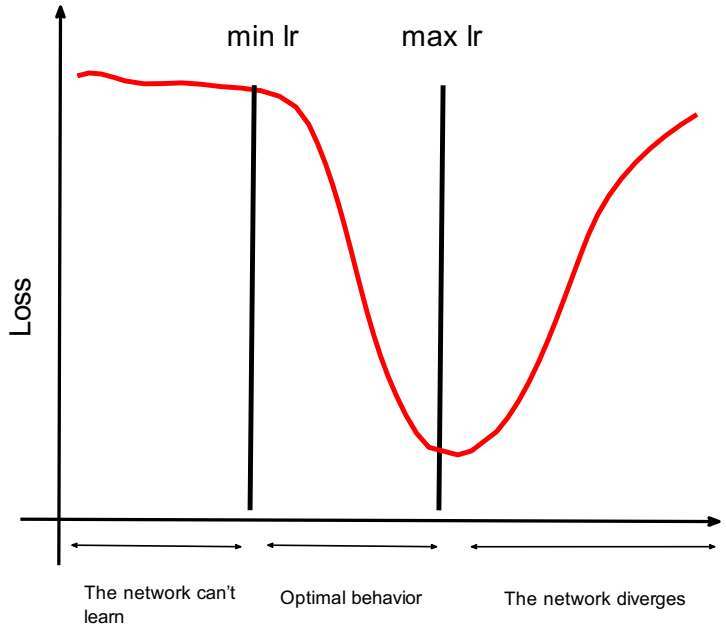


Example of triangular schedule

Deep Learning - Hyperparameters tuning - Triangular Schedule

Choosing the learning rate boundaries

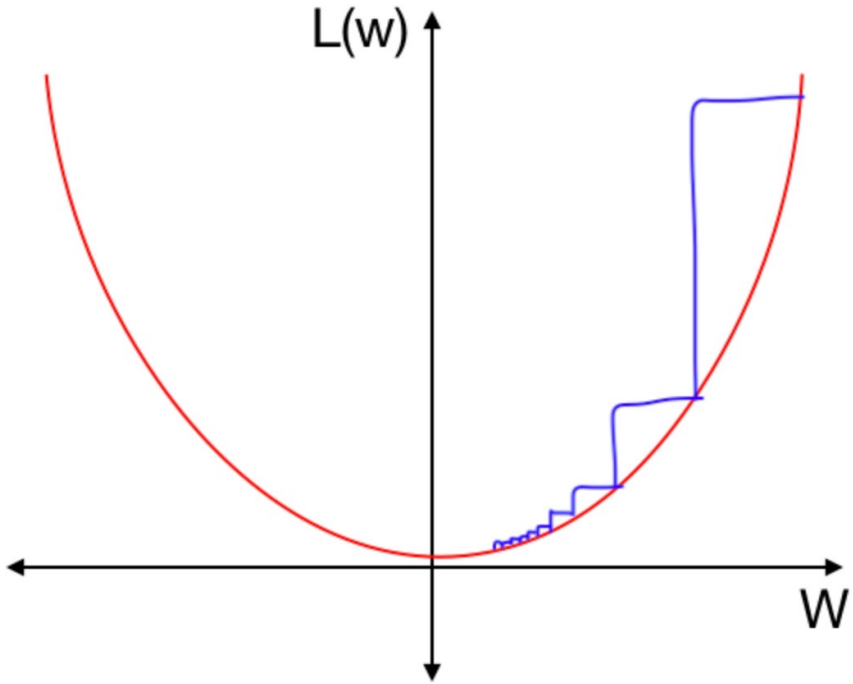
- We want to choose extreme values of learning rates for which the network is able to learn.
- Start very low (i.e. 10^{-5}) and augment the learning rate for one half cycle to a high value (i.e. 10^{-2}). Then choose with respect to the training curve



Search for learning rate boundaries

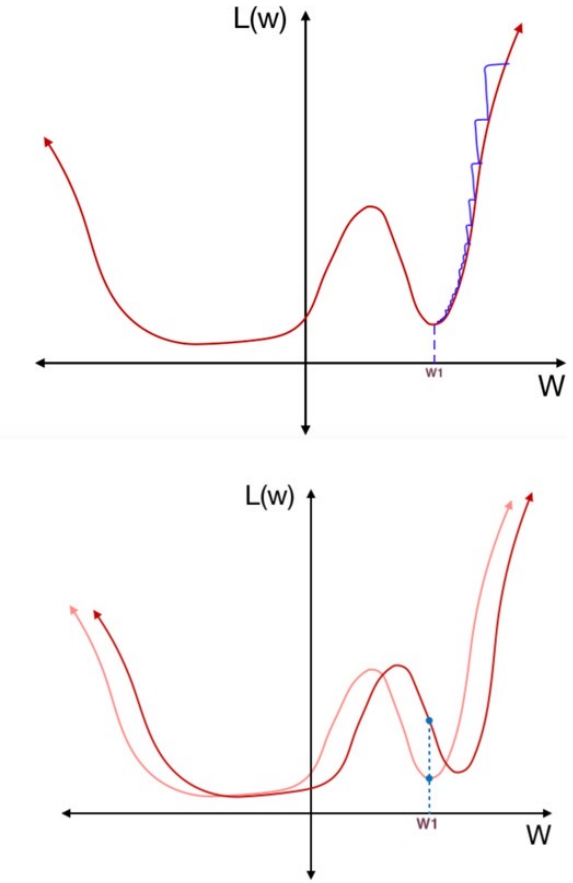
Deep Learning - Stochastic Gradient Descent With Restarts

- variant of learning rate annealing
- Gra dually decreases the learning rate through training
- start with relatively high learning rates for several iterations
- quickly approach a local minimum,
- then gradually decrease the learning rate as we get closer to the minimum



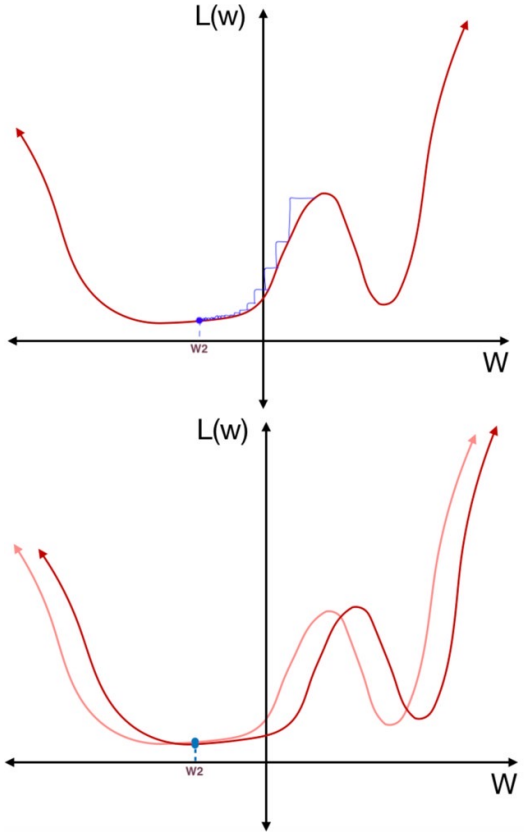
Deep Learning - Stochastic Gradient Descent With Restarts

- assume multiple minima
- small change in weights may result in a large change in loss.
- If test this network on a different dataset, loss function might be slightly different (small shift).
- small shift of the loss function in this local minimum (w_1) result in a large change of loss
- search for ”flatter” error valleys
- decreasing learning rate approached minimum



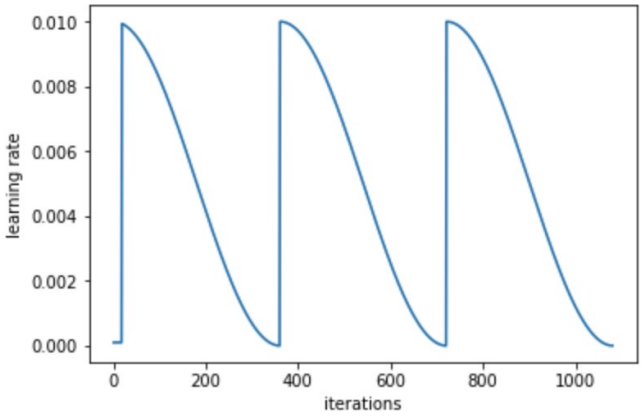
Deep Learning - Stochastic Gradient Descent With Restarts

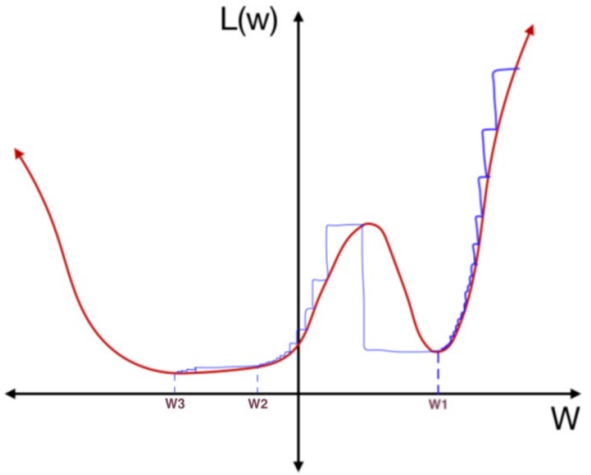
- search for ”flatter” error valley - w_2
- offers an accurate and stable solution- generalization
- better ability to react to new data.
- to find a more stable local minimum, we increase the learning rate from time to time, encouraging the model to “jump” from one local minimum to another if it is in a steep trough.
- SGDR - with restarts



Deep Learning - Stochastic Gradient Descent With Restarts

- In the first “half cosine” period in the graph above, we descend into a local minimum - w_1
- restart: increase learning rate
- gradually decrease learning rate again until minimize the loss function - w_1
- finding a stable solution



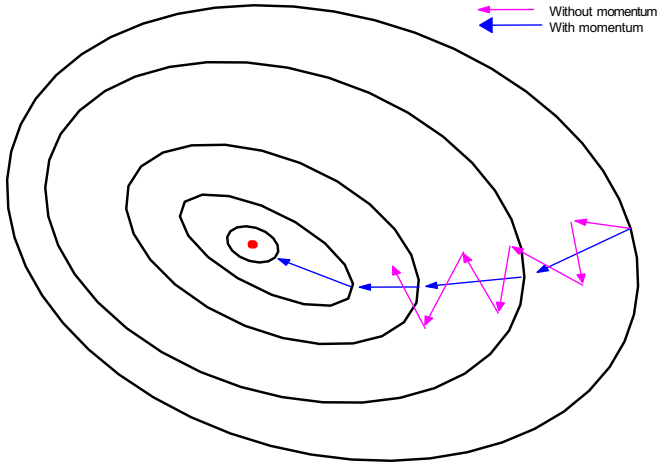


Deep Learning - Stochastic Gradient Descent With Momentum

- SGD is noisy, because submited to the data in the batch.
- To converge more smoothly and faster **momentum** keeps a memory of the previous update.

$$U_t = \beta U_{t-1} + (1 - \beta)\nabla_\theta \hat{L}_t$$

$$\theta_{t+1} = \theta_t - \eta U_t$$



Convergence with momentum

Deep Learning – RMSProp (Root Mean Square Propagation)

- Adaptative gradients, adapts the learning rate independently for every parameter, thus convergence is faster.
- Root Mean Square Propagation writes, for every parameter w:

$$v_t = \beta v_{t-1} + (1 - \beta) \, (\nabla_w \hat{L}_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla_w \hat{L}_t$$

  - v_t : second moment estimate/param
  - ε: small constant - numerical stability - prevent division by 0

Deep Learning - **ADAM (Adaptive Moment Estimation)**

ADAM combines

- **RMSprop** (adaptive learning rate based on second moments) and **Momentum** (which uses the exponentially decaying average of past gradients, or first moments).
- **bias correction** for the moment estimates, which is important during the initial steps of training.

ADAM Equations

Initialization: m_0=0(1st Moment Vector), v_0=0 (2nd Moment Vector), $\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

1.Update the First Moment Estimate (Momentum) m_i:

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t$$

m_t exponentially decaying average of past gradients, β_1 decay rate (~0.9).

2. Update Second Moment Estimate (RMSprop) v_t): $\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2$

β_1 decay rate (~0.9999)

3. **Perform Bias Correction** $\hat{\boldsymbol{m}}_t = \dfrac{\boldsymbol{m}_t}{1-\beta_1^t}$ and $\hat{\boldsymbol{v}}_t = \dfrac{\boldsymbol{v}_t}{1-\beta_2^t}$

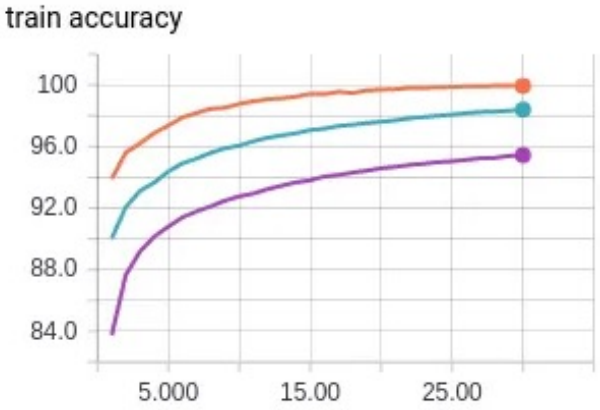
Update the parameters: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \dfrac{\eta}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon}\hat{\boldsymbol{m}}_t$

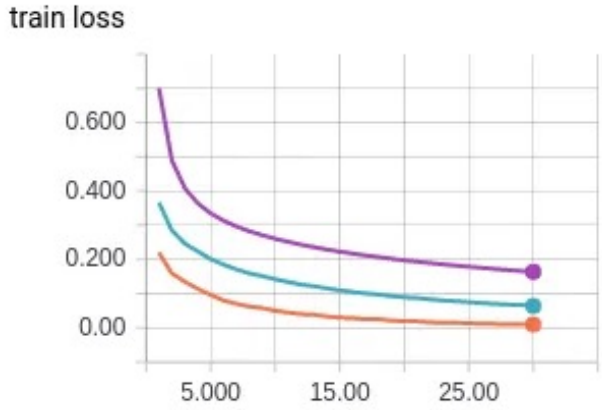
Deep Learning - Regularization - Batch size

- Batch: number of samples per gradient update.
- Rationale Noise is a shield against overfitting.
- the smaller the batch, the more noisy the convergence. deacreasing *batch size* regularizes the network.
- Small batches → noisy gradients → better generalization.
- Large batches → stable updates → faster on GPU but risk overfitting.
- batch size: trade off between regularization and computation time

Practical rule: start small (32–128) and scale up if needed.

- Orange : batch size 64
- Blue : batch size 256
- Purple : batch size 1024

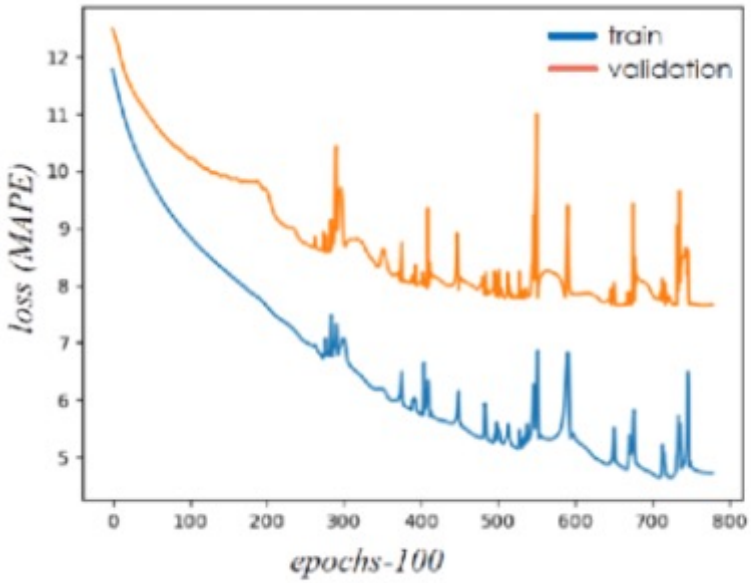




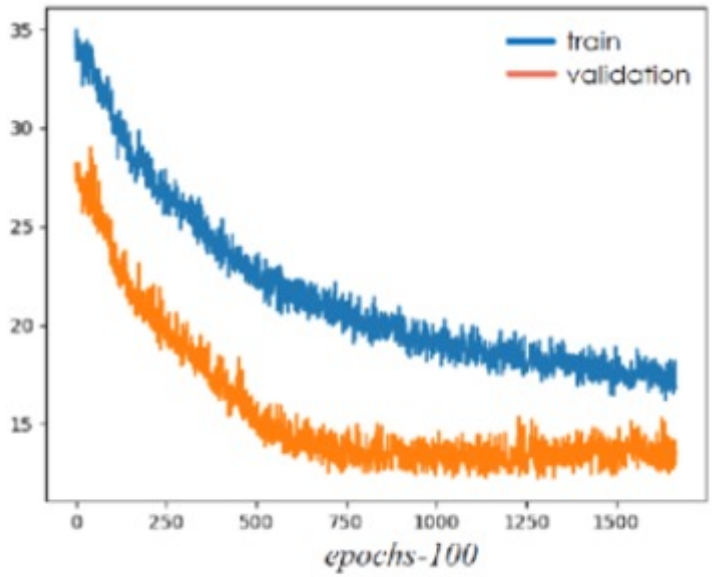
https://medium.com/mini-distill/effect-of-batch-size-on-training-dynamics-21c14f7a716e

Deep Learning - Regularization - Dropout

- Dropout, randomly selected neurons are temporarily ignored (dropped out) during training - sets to 0 a proportion of the input of a layer, Typical rates: 0.1–0.5 for dense layers.
- At each step of training, a new subnetwork is selected.
- Result an adaptation appears in the final network only if it exists in a sufficient part of the training data.
- Another interpretation of dropout: large numbers of subnetworks are learned and aggregated at test time.



without dropout



10% dropout

Bakas, Nikolaos & Markou, George & Charmpis, Dimos & Hadjiyiannakou, Kyriakos. (2021). PERFORMANCE AND SCALABILITY OF DEEP LEARNING MODELS TRAINED ON A HYBRID SUPERCOMPUTER: APPLICATION IN THE PREDICTION OF THE SHEAR STRENGTH OF SLENDER RC BEAMS. 10.7712/120121.8754.19593.

Deep Learning - Regularization - Dropout

difference between training (dropping neurons) and inference (scaling weights).: difference lies in maintaining the **expected output magnitude** of a neuron between the training phase and the inference (or test) phase.

- goal of dropout: randomly deactivate neurons during training to prevent co-adaptation and improve generalization.
- cannot simply deactivate neurons during inference=> introduce randomness and instability to final prediction.

Phase	Standard Dropout	Inverted Dropout (Preferred)
Training	Drop neurons by probability p.	Drop neurons by p, **scale remaining activations** by $\frac{1}{1-p}$.
Inference	Use all neurons, **scale weights** by $(1-p)$.	Use all neurons, **do nothing** (no scaling needed).

Deep Learning - Regularization - Weight decay

- Regularization of a neural network can also be standard
- L_2-constraint called weight decay.
- corresponds to adding the following penalty to the loss $\lambda \sum_w w^2$.
- Then weight updates:

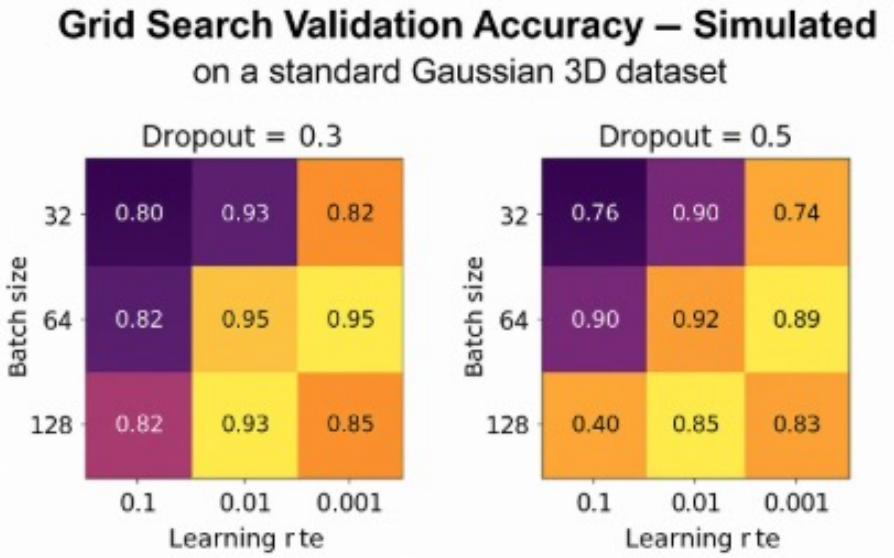
$$w_{t+1} = w_t - \eta \nabla_w \hat{L}_t - 2\lambda w$$

Grid Search

- exhaustive search over predefined hyperparameter values.

Example grid:

- learning_rate: [0.1, 0.01, 0.001], batch_size: [32, 64, 128], dropout: [0.3, 0.5] - combinations = 3 × 3 × 2 = 18 experiments.



- each model/combination typically evaluated using **k-fold Cross-Validation** (CV) - result is robust and not specific to a single train/validation split.
- **Pros:** Simple to implement, guarantees finding absolute best combination **within the defined grid**.
- **Cons:** Computationally **expensive** (exponentially scales with the number of hyperparameters and values), often misses the *true* optimum.

Concclusion

THANK YOU!

QUESTIONS?